

Планировчици и алгоритми — 116

1. Какво е планировчик (Scheduler)

Планировчикът е компонент на операционната система, който решава кой процес да работи следващ. Той е като диригент в оркестър - координира кога всеки "музикант" (процес) да свири.

Как работи планировчикът:

Процеси в опашка



Планировчик



CPU

Задачи на планировчика:

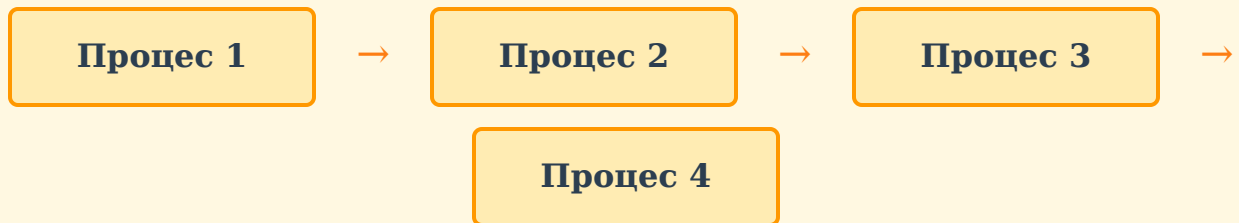
- **Избор на процес** - кой да работи следващ
- **Справедливост** - всички процеси да получат шанс
- **Ефективност** - максимално използване на CPU
- **Приоритети** - важните процеси първи

2. FCFS - First Come First Served

Какво е FCFS:

Най-простият алгоритъм - кой дойде първи, работи първи. Като опашка в магазина - първият клиент се обслужва първи.

Как работи FCFS:



Предимства

- Много прост
- Справедлив
- Лесно за разбиране
- Няма изглаждане на процеси

Недостатъци

- Дълго средно време на чакане
- Не е оптимален
- Кратък процес може да чака дълъг
- Не се отчита приоритетът

Пример с FCFS:

Процеси: A(8ms), B(4ms), C(2ms), D(6ms)

Ред: A → B → C → D

Средно време на чакане: $(0+8+12+14)/4 = 8.5\text{ms}$

3. SJF - Shortest Job First

Какво е SJF:

- **Най-краткият процес първи** - оптимизира времето
- **Предварително планиране** - знае колко време отнема
- **Минимално средно време** - теоретично най-добър
- **Проблем с дългите процеси** - могат да чакат завинаги

Как работи SJF:

Сортирай по време



Най-кратък първи



Следващият най-кратък

Пример с SJF:

Процеси: A(8ms), B(4ms), C(2ms), D(6ms)

Сортирани: C(2ms), B(4ms), D(6ms), A(8ms)

Ред: C → B → D → A

Средно време на чакане: $(0+2+6+12)/4 = 5\text{ms}$

4. Round Robin (RR)

Какво е Round Robin:

Всеки процес работи малко време (квант), после отива накрая на опашката. Като игра на "горещи картофи" - всеки държи малко време.

Round Robin опашка:

Процес А

Процес В

Процес С

Процес D



CPU (квант време)



Обратно в опашката

Параметри на Round Robin:

- **Квант време** - колко време работи всеки процес (10-100ms)
- **Преминаване** - ако процесът не завърши, отива накрая
- **Справедливост** - всички получават равен шанс
- **Отзивчивост** - добро за интерактивни приложения

Пример с Round Robin (квант=4ms):

Процеси: A(8ms), B(4ms), C(2ms), D(6ms)

Ред: $A(4) \rightarrow B(4) \rightarrow C(2) \rightarrow D(4) \rightarrow A(4) \rightarrow D(2)$

Всички процеси получават шанс да работят

5. Priority Scheduling

Какво е Priority Scheduling:

- **Всеки процес има приоритет** - число (1-10, 1-100)
- **По-ниско число = по-висок приоритет** (или обратно)
- **Приоритетът може да се променя** - динамично планиране
- **Проблем с изглаждане на процеси** - ниските приоритети могат да не работят

Как работи Priority Scheduling:

Сортирай по приоритет



Най-висок приоритет първи



Следващият по приоритет

Примери за приоритети:

- **Системни процеси** - приоритет 1 (най-висок)
- **Потребителски процеси** - приоритет 5 (среден)
- **Фонове задачи** - приоритет 10 (най-нисък)
- **Реално време** - приоритет 0 (критичен)

6. Сравнение на алгоритмите

Сравнителна таблица:

Алгоритъм	Средно време на чакане	Справедливост	Сложност	Изгладване на процеси
FCFS	Високо	Добра	Ниска	Няма
SJF	Най-ниско	Лоша	Средна	Има
Round Robin	Средно	Отлична	Ниска	Няма
Priority	Зависи	Зависи	Средна	Има

7. Кога се използва всеки алгоритъм

FCFS

- Прости системи
- Batch обработка
- Когато справедливостта е важна
- Образователни цели

SJF

- Batch системи
- Когато знаем времето на изпълнение
- Оптимизация на производителността

- Серверни приложения

Round Robin

- Интерактивни системи
- Desktop операционни системи
- Time-sharing системи
- Когато отзивчивостта е важна

Priority

- Реално време системи
- Когато има важни задачи
- Системи с различни типове процеси
- Критични приложения

8. Проблеми с планирането

Изгладване на процеси (Starvation):

Когато процес с нисък приоритет никога не получава шанс да работи. Като човек в опашка, който винаги се прескача.

Решения за изгладване на процеси:

- **Aging** - увеличаване на приоритета с времето
- **Round Robin с приоритети** - комбинация от двата
- **Максимално време на чакане** - принудително повишаване

- **Многониивони опашки** - различни опашки за различни приоритети

9. Заключение

Планировчиците са критични за производителността на операционната система. Няма един "най-добър" алгоритъм - всеки има своите предимства и се използва в различни ситуации.

Запомни:

- **FCFS** - прост и справедлив, но не оптимален
- **SJF** - оптимален, но може да има изглаждане на процеси
- **Round Robin** - справедлив и отзивчив
- **Priority** - гъвкав, но може да има изглаждане на процеси
- **Изглаждане на процеси** - проблем който трябва да се решава
- **Изборът** зависи от типа система и изискванията